

Laboratório 03 — formular como busca

D2 — Fundamentos de Agentes Inteligentes · Encontro 2, bloco 3 (tarde) Pacote: `D2-03-formular-como-busca.zip` · 30 minutos, os dois caminhos com o mesmo prazo

O que você já tem, e o que falta

Você já tem	O que falta
O PEAS (lab-01) e as sete dimensões (lab-02) do seu domínio	Um subproblema do seu domínio escrito nos cinco componentes
Os três subproblemas do catálogo, em código funcionando, em <code>comum/problemas/</code> — cada um com dados de exemplo comentados mostrando onde trocar	O teste de sanidade passando sobre a SUA formulação
O motor de busca (<code>comum/busca.py</code>), que conhece algoritmos e não conhece problema nenhum	A formulação registrada no <code>agent-spec.md</code> , seção 7
O <code>esqueleto.py</code> rodando com dados de exemplo mínimos — seis verificações verdes de fábrica	

Resolver vem na sexta que vem (lab-04); hoje é só formular. A formulação vale mais que o algoritmo — os algoritmos já estão prontos no motor.

Os dois caminhos

- **Padrão:** escolha UM dos três subproblemas do catálogo e troque os dados pelos do seu domínio. O `solucao.py` do pacote é o modelo da troca, linha a linha — ele adapta o subproblema 1 para o domínio dos pedidos travados.
- **Desafio:** formule o subproblema do seu domínio diretamente, nos cinco componentes — parta do `esqueleto.py`.

Trocar de caminho é livre, nos dois sentidos e sem justificar. Quem seguir o desafio e travar mais de dez minutos na formulação própria migra para o catálogo sem cerimônia — a troca não custa nota.

Os passos

1. Extraia o `D2-03` na raiz do repositório e leia o enunciado dos três subproblemas do catálogo (tabela do slide 45: ordem de verificações, sequência de consultas, roteiro de conformidade).
2. Caminho padrão: escolha um dos três e troque os dados. Desafio: formule o seu, nos cinco componentes, dentro do `esqueleto.py`.
3. Rode o teste de sanidade que vem no pacote — e é aqui que o laboratório se distingue: o `verificar-formulacao.py` confere SEIS propriedades, com mensagem que diz o que fazer em cada falha:

'''

```
python verificar-formulacao.py meu_problema.py:MEU_PROBLEMA
```

...

As seis: estado hashable; transição que é função; ações estáveis na ordem; custo positivo ou zero; objetivo alcançável; e o erro do estado que guarda o caminho — duas trilhas diferentes até a mesma situação precisam ser o mesmo estado. Rode sem argumentos para vê-lo funcionar sobre o catálogo inteiro.

1. Registre a formulação no `agent-spec.md`, seção 7 — é o item 3 da Atividade Orientada, e a entrada dos dois laboratórios de sexta.

Critério de aceite

Os cinco componentes preenchidos e o teste de sanidade passando — as seis verificações verdes sobre a sua formulação.

Quanto tempo isto deve levar

Trinta minutos. O esqueleto já roda de fábrica; quem adapta do catálogo leva menos tempo que quem formula do zero — e é para isso que ele existe.

Se travar

1. **Não sabe por onde começar?** Comece pelo teste de objetivo — é ele que define o que precisa estar no estado, e os outros quatro componentes saem dele.
2. **As ações se multiplicaram?** Quase sempre há um grupo que produz o mesmo efeito e cabe numa ação só — junte-as. Ações demais são o erro de granularidade mais comum.
3. **O teste de sanidade acendeu o aviso da verificação 6?** O estado está guardando a história do caminho. Guarde a situação: duas trilhas que chegam ao mesmo resultado devem produzir o mesmo estado — o caminho já fica guardado nos nós da busca. Foi este erro que o material da própria disciplina cometeu uma vez, no subproblema de consultas: o espaço inflou de 1 500 para 117 mil expansões até o teste pegar.
4. **Tente resolver?** Não — hoje é só formular. Quem tenta rodar busca agora gasta os trinta minutos depurando o que ainda nem precisa rodar.